

CSCE 605 COMPILER DESIGN · MODULE 09 · LECTURE NOTES

Optimisation

A search with no gradient, settled by measurement.

MODULE OUTCOMES

- Implement the standard local and global optimisations.
- Explain loop optimisations and why loops dominate.
- Explain the phase-ordering problem.
- Explain inlining as the enabling transformation.
- Measure a pass honestly, including when it loses.

1 The catalogue

Pass	What it does	What it needs
Constant folding	Evaluate expressions over known constants at compile time.	Nothing. The cheapest possible win, and it runs constantly.
Constant propagation	Replace a variable by its known constant value at each use.	SSA (Module 07) or reaching definitions (Module 08).
Dead code elimination	Delete computations whose results are never used. Iterates, because deleting one use can make its operands dead.	Liveness, or in SSA simply a use count of zero.
CSE and global value numbering	Reuse a value already computed rather than recomputing it. GVN is the SSA-based, stronger form.	Available expressions, or SSA value numbering.
Copy propagation	Replace uses of a copy with its source, so the copy becomes dead.	SSA. Mostly cleanup after other passes.
Inlining	Replace a call with the callee's body.	A call graph. The enabling transformation — §3.
Strength reduction	Replace an expensive operation with a cheaper equivalent — a multiply by an induction variable becomes a running addition.	Loop structure and induction variable recognition.
Loop-invariant code motion	Hoist computations that do not change out of the loop.	Loop structure plus dominance — the hoisted code must execute on every path that the loop body did.

These interact, and the interaction is the whole point. Inlining substitutes a constant argument, which lets constant propagation fire, which makes a branch condition constant, which lets the branch fold, which makes half the body unreachable, which dead code elimination removes — and the smaller remaining function may now itself be worth inlining somewhere else. **Individually these passes are unimpressive; the cascade is what produces the difference between -O0 and -O2.**

LOOPS ARE WHERE THE TIME IS

Programs spend nearly all their time in loops (CSCE 735 Module 07), so any transformation applied inside a loop is multiplied by the trip count. A one-instruction saving in a loop running a million times is a million instructions.

Loop-invariant code motion hoists computations whose operands do not change within the loop — one evaluation instead of n . **The subtlety is that the hoisted code must be guaranteed to execute**, or you have introduced a computation (and possibly a fault) that the original program would not have performed.

Strength reduction replaces an expensive operation with a cheaper one: an address computation $\text{base} + i*4$ becomes a pointer incremented by 4 each iteration, turning a multiply into an add.

Unrolling amortises the loop overhead — the increment, the compare, the branch — over several iterations, and exposes instruction-level parallelism for the out-of-order engine to exploit (CSCE 614).

Vectorisation then uses the SIMD units (CSCE 735 Module 02), and when it applies it is the single largest win available — $4\times$ to $16\times$ on the arithmetic — which is why so much compiler engineering goes into making it apply.

2 Phase ordering

PASSES ENABLE AND DISABLE EACH OTHER

Running pass A before pass B can expose opportunities for B, or destroy them. There is no general rule, and the effect is program-dependent.

Inlining before constant propagation lets the caller's constant arguments become constants inside the body. **Running it after** misses that entirely — and it is the single most valuable interaction in the whole pipeline.

But inlining increases code size, which can degrade instruction cache behaviour and make the program *slower* (CSCE 614) — so even a pass that is clearly beneficial in isolation has a cost that only shows up in interaction with the hardware.

There is no ordering optimal for all programs, and finding the optimal ordering for a single given program is intractable — the space is the permutations of dozens of passes with repetition allowed. **So real compilers use a hand-tuned sequence with passes repeated**, arrived at empirically over years and defended by benchmark suites rather than by argument.

LLVM -O2, heavily abridged:

```
SROA                (memory -> SSA registers; FIRST, because
                    everything downstream needs SSA values)
early CSE
simplify CFG
instcombine         (peephole over IR)
inline              (the enabler)
SROA again          (inlining exposed more)
instcombine again
jump threading
LICM
loop unroll / vectorise
GVN
dead store elimination
simplify CFG again
```

SROA, instcombine and simplify-CFG each appear SEVERAL times.
The repetition is the admission that ordering has no clean answer.

Run ``opt -print-after-all`` once and read the whole thing.

3 Inlining

INLINING IS VALUABLE FOR WHAT IT ENABLES

Removing the call overhead is the minor benefit. A direct call on a modern machine is a handful of cycles, perfectly predicted, and overlapped with other work. If that were all inlining bought, it would be a marginal pass.

The real value is that it makes the callee's body visible to every other optimisation, with the caller's actual arguments substituted. The body stops being an opaque call and becomes ordinary code in context.

So a constant argument becomes a constant inside the body, a branch on it folds away, half the body becomes unreachable and is deleted, loop bounds may become known, and the much smaller remainder may be worth inlining further up the chain.

Inlining is the transformation that unlocks the others, and this explains several things at once: why C++ templates and Rust generics are fast (**monomorphisation is inlining with the types substituted too**, so the generic code specialises completely), why virtual calls are expensive beyond their direct cost (they cannot be inlined without devirtualisation), and why link-time optimisation exists (it extends inlining across translation units).

Signal	Effect on the decision	Note
Callee size	Small bodies are inlined.	The dominant signal in every real heuristic , usually measured in IR instructions after a cheap simplification.
Called exactly once	Always inline.	There is no size cost — the original is deleted afterwards. Free.
Constant arguments at the call site	Strongly favour.	Precisely because of what it enables , per the callout above. LLVM estimates the simplification that would result.
Call site is inside a loop	Favour.	The benefit is multiplied by the trip count (\$1).
Recursive	Inline to a bounded depth, or not at all.	Unbounded recursive inlining does not terminate.
Profile says the call site is cold	Do not inline.	Code size costs instruction cache everywhere , including in the hot code that has nothing to do with this call. The cost is non-local, which is what makes this hard.

4 Measuring

MEASURE EACH PASS SEPARATELY, AND REPORT THE LOSSES

Enable one pass at a time against a fixed baseline, over a benchmark suite with real programs in it — not three microbenchmarks that happen to be in the repository.

Some passes will lose on some programs. That is normal and it is information. **Hiding it is how compilers accumulate passes that do nothing** — a pass added for one benchmark in 2009, never re-measured, still running on every compile.

Report the distribution rather than the mean (CSCE 735 Module 07). A pass that improves the average by 10% because it improves one benchmark by 200% and nothing else is a completely different thing from one that improves everything by 10%, and the mean cannot distinguish them.

And measure code size and compile time alongside speed. A 2% speedup bought with 40% more compile time is usually a bad trade — compile time is developer time, multiplied by every build — and nobody notices unless it is measured deliberately.

- **Dead code elimination removes the benchmark.** If nothing consumes the result, the entire loop is deleted and you measure an empty function. Consume the result, or use a compiler barrier.
- **Constant folding evaluates the whole computation at compile time** if the inputs are literals, and you measure a return statement. Read the inputs from somewhere opaque.
- **Code layout changes shift performance by several percent** for reasons entirely unrelated to the pass being measured — a hot loop crossing a cache line boundary differently, or an alignment change. **This effect is large, underappreciated, and can exceed the optimisation you are measuring;** the published work on it (Mytkowicz et al.) showed link order alone moving results by more than the optimisations under study.
- **Three microbenchmarks is not a suite.** Passes get tuned to whatever is measured, so a small suite produces a compiler that is good at that suite.
- **And compare against -O2, never against -O0** — CSCE 735 Module 07's optimised baseline rule, applied here. A speedup over unoptimised code is not a result.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Cooper & Torczon — Engineering a Compiler, chapters 8 and 10	The catalogue of §1 with the correctness conditions for each transformation stated.
LLVM — passes documentation, and opt -print-after-all (free)	Run it on your own code this week. Watching the IR change pass by pass is worth more than any description of the passes.
Chandler Carruth — talks on LLVM optimisation and benchmarking (free)	Particularly the benchmarking material of §4 , including how microbenchmarks mislead and what to do instead.

Resource	What it covers for this module
Mytkowicz, Diwan, Hauswirth & Sweeney — Producing Wrong Data Without Doing Anything Obviously Wrong (free)	The code layout effect of §4 , measured. Link order and environment size alone moved results more than the optimisations being studied.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement constant folding, constant propagation, and dead code elimination over your SSA IR.
- 2 Implement global value numbering and measure how many redundant computations it removes.
- 3 Implement LICM. **Construct a case where naive hoisting introduces a fault** the original program would not have had.
- 4 Implement inlining with a size-based heuristic.
- 5 **Demonstrate the cascade**: show a function where inlining alone does little but inlining followed by three other passes transforms it.
- 6 **Run your passes in two different orders** and measure the difference on each benchmark.
- 7 Run `opt -print-after-all` on a C file and identify five transformations in the output.
- 8 **Measure each of your passes separately** and produce the table, including the negative results.
- 9 **Write a benchmark that dead code elimination deletes**, observe the impossible result, then fix the benchmark.
- 10 Measure compile time and code size alongside speed, and find a pass whose trade is bad.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name eight optimisations and what each requires.
- 2 Describe the cascade and give an example chain.
- 3 Why do loops get their own category of optimisation?
- 4 State the phase-ordering problem and how real compilers respond.
- 5 Why is inlining valuable, and what does that explain about generics?
- 6 Give six inlining signals and say why code size is a real cost.
- 7 How should a pass be measured and reported?
- 8 Name five ways optimisation benchmarks lie.

